

Catalogue des fonctions backend

Plateforme d'intelligence de marché (inspirée de Statista et Mordor Intelligence)

- Liste détaillée des modules backend, des fonctions (services), des endpoints REST et des règles métier.
- Inclut : paywall/lead magnet (preview + floutage/redacted), achats à l'unité, abonnements (simple/premium), dashboard premium, veille économique, services corporate, back-office, import/export SQL.
- Date de génération : 10/02/2026.

Base documentaire : cahier des charges complet + cahier des charges simplifié fournis par le porteur de projet.

Sommaire

- 1. Principes de conception backend
- 2. Modules backend et fonctions (A à N)
- 3. Modèle de données minimal
- 4. Règles métier clés
- 5. Tâches planifiées (cron/jobs)
- 6. Phasage de livraison recommandé
- 7. Sources

1. Principes de conception backend

Le backend est la couche responsable de la logique métier, de la sécurité, de l'accès aux données (statistiques, rapports, infographies) et de l'exposition d'une API (REST ou GraphQL) au frontend.

Dans ce projet, le backend doit aussi gérer un modèle d'accès différencié (abonnement, achat à l'unité, quotas) et un paywall/lead magnet (aperçus gratuits et pages floutées/redacted).

Recommandation : centraliser l'autorisation (RBAC/ABAC) et les droits d'accès (entitlements) côté serveur afin que le frontend ne puisse jamais contourner le paywall.

Principes recommandés :

- API sécurisée : authentification, rôles, contrôle d'accès par ressource (rapport, page, dataset, export).
- Performance : cache et pré-calculs (dashboard), index de recherche, diffusion par pages rendues (viewer).
- Évolutivité : architecture modulaire (monolithe modulaire ou microservices) avec séparation des domaines (contenu, billing, search, viewer).
- Traçabilité : audit log (lectures, achats, actions admin), analytics d'usage et de recherche.

2. Modules backend et fonctions (A à N)

Chaque module ci-dessous est présenté avec : objectif, fonctions backend (services), endpoints REST recommandés et notes d'implémentation.

A. Authentification et sécurité (Auth / IAM)

Objectif : Sécuriser l'accès à la plateforme, gérer les rôles et droits, et tracer les actions sensibles.

Fonctions backend :

- registerUser() / createUser() : création de compte (self-signup ou création par admin).
- login(), logout(), refreshToken() : gestion de session (JWT/cookies sécurisés).
- forgotPassword(), resetPassword() : récupération de compte.
- verifyEmail() (recommandé) et 2FA (optionnel) : renforcement de la sécurité.
- assignRole(), revokeRole(), checkPermission() : RBAC/ABAC pour protéger chaque ressource.
- auditLog(event) : journalisation (auth, lecture, achats, exports, actions admin).

Endpoints REST (exemples) :

```
POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
POST /auth/forgot-password
POST /auth/reset-password
```

Notes / règles :

- Chiffrer les données sensibles (hash des mots de passe, secrets, tokens).
- Limiter les tentatives de connexion (rate limiting) et activer des logs d'audit.

B. Profils, favoris et préférences (User Profile)

Objectif : Personnaliser l'expérience (secteurs suivis, favoris, historique de recherche/lecture).

Fonctions backend :

- getProfile(), updateProfile() : informations compte et organisation.
- setFavorites(), getFavorites() : secteurs, thématiques et rapports favoris.
- saveSearchHistory(), getSearchHistory() : historique de recherche (opt-in selon RGPD).
- setNotificationPreferences() : newsletter/alertes par secteurs.
- getViewHistory(), getDownloadHistory() : traçabilité par plan (si téléchargement activé).

Endpoints REST (exemples) :

```
GET /me
PATCH /me
GET /me/favorites
POST /me/favorites
GET /me/search-history
DELETE /me/search-history
```

Notes / règles :

- Prévoir un mécanisme d'opt-out pour l'historique et les notifications.

C. Catalogue de contenus (Rapports, statistiques, infographies)

Objectif : Gérer la création, la publication et la mise à jour périodique des contenus et de leurs métadonnées.

Fonctions backend :

- createReport(), updateReport(), publishReport(), archiveReport() : cycle de vie des rapports.
- uploadReportFile(file) : ingestion des fichiers (PDF en MVP; autres formats selon forfait).
- createReportVersion() : versioning (mise à jour trimestrielle/semestrielle).
- setMetadata(report) : secteur, géographie, période, tags, résumé, mots-clés.
- createIndicator(), updateIndicator() : indicateurs clés extraits des rapports.
- attachAssets(reportId, charts, tables, images) : lier visuels et données au rapport.

Endpoints REST (exemples) :

```
GET /contents?type=report|stat|infographic&sector=&geo=&period=&q=
GET /reports/:id
GET /reports/:id/metadata
```

Notes / règles :

- Normaliser les référentiels (secteurs, géographies, périodes) pour des filtres cohérents.
- Prévoir une validation éditoriale avant publication.

D. Moteur de recherche et filtres (Search)

Objectif : Recherche unifiée type Statista/Mordor : full-text, facettes, suggestions, tri.

Fonctions backend :

- search(query, filters, sort, page) : recherche principale (texte + filtres).
- autocomplete(prefix) : suggestions instantanées (mots-clés, secteurs, pays).
- getFacets(query) : comptages par facettes (secteur, pays, période, type).
- indexContent(), reindexContent() : indexation après import/publish.
- trackSearchAnalytics() : requêtes populaires et amélioration SEO/conversion (admin).

Endpoints REST (exemples) :

```
GET /search?q=&filters...&sort=&page=
GET /search/autocomplete?prefix=
GET /search/facets?q=&filters...
```

Notes / règles :

- Compatibles avec une base relationnelle + moteur full-text (Postgres FTS) ou un moteur dédié (OpenSearch/Elastic) selon charge.

E. Paywall et lead magnet (preview, floutage, extraits)

Objectif : Montrer un échantillon gratuit et verrouiller le contenu premium selon l'abonnement ou l'achat.

Fonctions backend :

- `getReportAccessScope(user, reportId)` : détermine exactement ce qui est visible (pages, graphiques, export).
- `generatePreview(reportId, rules)` : pack preview (résumé, table des matières, pages visibles).
- `getReportPage(reportId, pageNumber, accessScope)` : renvoie page nette ou floutée/redacted.
- `getReportExcerpt(reportId)` : extrait téléchargeable (si stratégie lead magnet).
- `watermark(report, user)` : watermark dynamique (anti-partage) (optionnel).

Endpoints REST (exemples) :

```
GET /reports/:id/preview
GET /reports/:id/pages/:pageNumber
```

Notes / règles :

- Implémenter un « feature flag » pour activer/désactiver le téléchargement selon le plan.
- Pour le redacted : préférer un rendu image (pages) plutôt que servir le PDF texte intégral.

F. Lecteur de rapport sécurisé (Viewer service)

Objectif : Permettre la consultation exclusivement sur le site, avec contrôle anti-hotlink et session de lecture.

Fonctions backend :

- `createReadingSession(user, reportId)` : token de lecture à durée courte.
- `getSignedAssetUrl(assetId, sessionToken)` : URL signée pour assets (pages rendues, images).
- `streamPdf(reportId)` ou `servePageTiles(reportId, page)` : diffusion contrôlée (lecture seule).
- `config` : plein écran exclusif, sélection/copie (selon règles du produit).

Endpoints REST (exemples) :

```
POST /viewer/sessions (optionnel)
GET /reports/:id/pages/:pageNumber
GET /reports/:id/assets/:assetId
```

Notes / règles :

- Le backend doit décider si le viewer reçoit une page image (sans copie) ou une couche texte autorisée.

G. Abonnements, achats unitaires, quotas et facturation (Billing / Entitlements)

Objectif : Gérer les plans (PME, chercheurs, institutions, premium), achats à l'unité, quotas, factures et droits d'accès.

Fonctions backend :

- `createPlan()`, `updatePlan()`, `getPlanEntitlements()` : matrice de droits par plan.
- `startSubscription()`, `cancelSubscription()`, `changePlan()` : cycle de vie abonnement.

- `purchaseReport(reportId)` : achat à l'unité avec accès illimité (lecture).
- `consumeQuota(user, action)`, `getQuotaStatus()` : limitations (nb rapports, exports).
- `createInvoice()`, `downloadInvoice()`, `recordPayment()`, `paymentWebhookHandler()` : facturation et paiements.

Endpoints REST (exemples) :

```
GET /plans
POST /subscriptions
PATCH /subscriptions/:id
DELETE /subscriptions/:id
POST /purchases/reports/:reportId
GET /me/entitlements
GET /me/invoices
GET /invoices/:id/download
```

Notes / règles :

- Le backend doit être la « source of truth » pour les droits (entitlements), jamais le frontend.

H. Promotions, offres flash et codes promo

Objectif : Piloter des remises et bannières marketing (durée limitée) via le back-office.

Fonctions backend :

- `createPromotion()`, `updatePromotion()`, `expirePromotion()` : gestion des campagnes.
- `createCoupon(code, rules)`, `validateCoupon()`, `applyCoupon()` : codes promo et règles d'éligibilité.
- `listActivePromotions(placement)` : home, sidebar, popup.
- `trackPromotionConversions()` : suivi conversion (admin).

Endpoints REST (exemples) :

```
GET /promotions/active?placement=home|sidebar|popup
POST /coupons/validate
POST /coupons/apply
```

Notes / règles :

- Limiter l'usage des coupons (usage max, dates, plans éligibles).

I. Dashboard premium et visualisations (Analytics delivery)

Objectif : Fournir des agrégats et séries temporelles pour tableaux de bord premium (comparaisons, tendances, alertes).

Fonctions backend :

- `getDashboardKpis(sector, geo, period)` : KPIs par secteur/pays/période.
- `getTimeSeries(indicatorId, filters)` : séries temporelles.
- `getComparisons(indicatorId, by=geo|sector)` : comparatifs.
- `getUpdateAlerts(user)` : contenu mis à jour (trimestriel/semestriel).
- `computeAggregates()` : pré-calcul batch pour performance.

Endpoints REST (exemples) :

```
GET /dashboard/kpis?sector=&geo=&period=
GET /dashboard/updates
GET /dashboard/comparisons?indicatorId=&by=geo|sector
```

Notes / règles :

- Prévoir cache + pré-calculs pour éviter des requêtes lourdes en temps réel.

J. Actualités et veille économique (News / Watch)

Objectif : Publier un fil d'actualités, des analyses et un agenda, avec veille personnalisée par secteurs.

Fonctions backend :

- createNewsArticle(), updateNewsArticle(), publishNewsArticle() : cycle de vie articles.
- listNews(filters) : secteur, tags, période.
- createEconomicEvent(), listEconomicCalendar() : agenda économique.
- subscribeToWatchlist(user, sectors), generateDigest(user) : veille et newsletters.

Endpoints REST (exemples) :

```
GET /news?sector=&from=&to=
GET /news/:id
GET /economic-calendar?from=&to=
POST /watchlist
GET /watchlist
```

Notes / règles :

- MVP : collecte manuelle/semi-auto (pas de génération IA complexe).

K. Services corporate (Custom research, devis, licences)

Objectif : Gérer les demandes B2B sur mesure : briefs, devis, suivi d'état, comptes entreprise multi-utilisateurs.

Fonctions backend :

- submitCustomResearchRequest(form), submitQuoteRequest(form) : formulaires de demande.
- uploadBriefAttachment(requestId, file) : pièces jointes.
- setRequestStatus(), assignAnalyst() : workflow (new -> in_review -> quoted -> in_progress -> delivered).
- createOrganization(), addOrganizationUser(), setSeatLimit(), trackSeatsUsage() : gestion entreprises.

Endpoints REST (exemples) :

```
POST /corporate/requests
POST /corporate/quotes
GET /corporate/requests/:id
POST /corporate/requests/:id/attachments
```

Notes / règles :

- Prévoir SLA/support prioritaire selon plan corporate.

L. Simulateur échantillon et marge d'erreur (outil méthodologique)

Objectif : Formulaire interactif (type SurveyMonkey) pour calculer taille d'échantillon, marge d'erreur et générer un guide.

Fonctions backend :

- `calculateMarginOfError(n, confidenceLevel, p)` : calcul statistique standard.
- `calculateSampleSize(populationSize, confidenceLevel, marginOfError, p)` : taille d'échantillon.
- `saveMethodologyBrief(user, answers)` : sauvegarde du brief.
- `generateMethodologyGuidePdf(briefId)` : document guide (option).
- `recommendMethodology(answers)` : recommandations simples (MVP) si IA activée.

Endpoints REST (exemples) :

```
POST /methodology/calculate-sample-size
POST /methodology/calculate-margin-of-error
POST /methodology/briefs
GET /methodology/briefs/:id
POST /methodology/briefs/:id/generate-guide
```

Notes / règles :

- Garder un fonctionnement déterministe pour les calculs; l'IA (si présente) ne doit pas altérer les résultats mathématiques.

M. Back-office administration (Admin Console)

Objectif : Permettre à l'équipe interne de gérer contenus, utilisateurs, plans, promotions et statistiques d'usage.

Fonctions backend :

- `adminImportContent(files, metadata)`, `adminValidateContent()` : import + validation éditoriale.
- `adminScheduleUpdate(contentId, frequency)` : planifier mises à jour.
- `adminBulkImportUsers(csv)`, `adminSetRoles()` : gestion utilisateurs.
- `adminManagePlans()`, `adminInvoices()`, `adminPaymentsOverview()` : facturation et plans.
- `adminUsageDashboard()` : trafic, lectures, conversions, exports, top recherches.

Endpoints REST (exemples) :

```
POST /admin/import
POST /admin/reports
PATCH /admin/reports/:id
POST /admin/reports/:id/publish
GET /admin/users
POST /admin/users/import
GET /admin/plans
GET /admin/usage
```

Notes / règles :

- Tout endpoint /admin doit être protégé par rôle admin/éditeur + audit log.

N. Import, export et compatibilité SQL (ETL / Data Integration)

Objectif : Assurer l'ingestion et l'export de données (SQL/Excel/XML/CSV) avec validation et normalisation.

Fonctions backend :

- `importFromExcel()`, `importFromCsv()`, `importFromXml()`, `importFromSqlDump()` : ingestion.
- `validateSchema()` : contrôle qualité (types, champs obligatoires).
- `deduplicateSources()`, `normalizeGeographies()`, `normalizeSectors()` : nettoyage/normalisation.
- `exportDataset(format)` : CSV/Excel/XML selon droits.
- `exportMetadataSql()` : exports compatibles SQL pour intégration.

Endpoints REST (exemples) :

POST `/admin/import` (ou `/etl/import`)

GET `/exports/dataset?format=csv|xlsx|xml` (si autorisé)

Notes / règles :

- Mettre en file (queue) les imports lourds et journaliser les erreurs de mapping.

3. Modèle de données minimal

Pour supporter les fonctions ci-dessus, le backend doit au minimum gérer les entités suivantes (noms indicatifs).

- User, Role, Permission, AuditLog
- Organization, OrgUser, Seat
- Plan, Subscription, Purchase, Entitlement, Quota, Invoice, Payment
- Report, ReportVersion, ReportAsset, ReportAccessRule
- Indicator, Dataset, ChartDefinition, Source
- Sector, Geography, Period, Tag
- NewsArticle, EconomicEvent, WatchlistSubscription
- Promotion, Coupon
- CustomResearchRequest, QuoteRequest, Attachment
- MethodologyBrief, MethodologyGuide
- UserFavorite, SearchHistory, ViewEvent, DownloadEvent

Remarque : le schéma exact (SQL) dépendra des choix techniques (PostgreSQL, MySQL, etc.), mais la séparation « contenu / recherche / entitlements / facturation / admin » est fortement recommandée.

4. Règles métier clés

Ces règles doivent être codées côté serveur (et testées), car elles conditionnent l'expérience produit et la sécurité du contenu premium.

- Accès contenu : non connecté -> teaser; connecté sans droits -> preview + pages floutées/redacted; acheteur -> accès illimité au rapport acheté (lecture web); abonné -> accès selon plan (quota, secteurs, dashboard, exports).
- Téléchargement : les documents mentionnent à la fois « téléchargement restreint » et « lecture uniquement ». Recommandation : implémenter un feature flag + entitlements (ex. `can_download_pdf`, `can_export_dataset`) pour activer/désactiver par plan sans refactor.
- Viewer : privilégier une diffusion en pages rendues (images/tiles) pour éviter l'extraction intégrale du contenu premium; activer la sélection/copie seulement si le plan le permet.
- Source of truth : droits/entitlements et quotas sont calculés côté backend; le frontend ne doit pas décider l'accès.

5. Tâches planifiées (cron/jobs)

Ces traitements asynchrones améliorent la performance, la qualité des données et l'automatisation des opérations (marketing, facturation, maintenance).

- Re-indexation du moteur de recherche après import/publication.
- Expiration automatique des promotions/offres flash/coupons.
- Génération et envoi des newsletters/alertes de veille.
- Génération de factures et rapprochement des paiements (webhooks PSP).
- Calcul d'agrégats du dashboard (quotidien/hebdomadaire) + cache.
- Nettoyage des sessions de lecture et rotation des tokens signés.
- Contrôle périodique des quotas (remise à zéro mensuelle si applicable).

6. Phasage de livraison recommandé

Découpage en lots pour livrer un MVP commercial viable puis enrichir progressivement.

- Lot 1 (MVP) : Auth + Catalogue + Search + Paywall preview + Viewer lecture seule + Achats/Abos + Admin contenus.
- Lot 2 (Premium) : Dashboard + indicateurs + comparaisons + alertes de mise à jour.
- Lot 3 (Growth) : Actus/veille + promotions/coupons + analytics conversion.
- Lot 4 (Corporate) : demandes sur mesure + organisations/sièges/licences.
- Lot 5 (Tooling) : simulateur échantillon/marge d'erreur + génération guide méthodologique.

7. Sources

Ce document est une synthèse technique basée sur les deux fichiers fournis :

- Cahier des charges : « Plateforme d'intelligence de marché » (document détaillé).
- Cahier des charges simplifié : « Plateforme Data & Market Intelligence ».

Les exigences fonctionnelles (recherche, paywall, abonnements, dashboard premium, veille, corporate, admin, import/export SQL) proviennent directement de ces cahiers des charges.